

Sumário

1	Introdução	2
1.1	A rede neural e suas camadas	2
1.2	Neurônio	2
2	Modelo matemático de output para a rede neural	4
2.1	Para um único neurônio	4
2.2	Para toda a camada	4
2.3	Para toda a rede	5
2.4	ReLU: Opção melhor do que Sigmoid	5
2.5	Camada final: Sem ReLU	6
2.5.1	Problema matemático (saturação) com sigmoide	6
3	Como a rede neural aprende	6
3.1	Custo do erro (módulo do vetor erro)	6
3.2	Achando peso ideal em $\mathbb{R}^1$	7
3.3	Achando peso ideal em $\mathbb{R}^n$ com Função Custo MSE	8
3.3.1	Introdução	8
3.3.2	Gradiente e gradiente descendente	8
3.3.3	Aplicando gradiente descendente à Função Custo MSE	11
3.3.4	Conclusão para última camada	12
3.3.5	Camadas anteriores	12
3.3.6	Resumo: Ajuste de peso em $\mathbb{R}^n$ (Função Custo: MSE)	13
3.3.6.1	Ajuste de peso para última camada	13
3.3.6.2	Ajuste de peso para hidden layers	13
3.4	Achando Bias ideal em $\mathbb{R}^n$ para Função Custo Qualquer	14
3.5	Achando Peso ideal em $\mathbb{R}^n$ para Função Custo qualquer	14

1 Introdução

1.1 A rede neural e suas camadas

A rede neural consiste em um aglomerado de neurônios divididos por camadas. Em cada camada, cada neurônio recebe como input todas as saídas dos neurônios anteriores, aplicando pesos.

No exemplo, estamos tentando reconhecer o número 9. De modo teórico, pode-se assumir que:

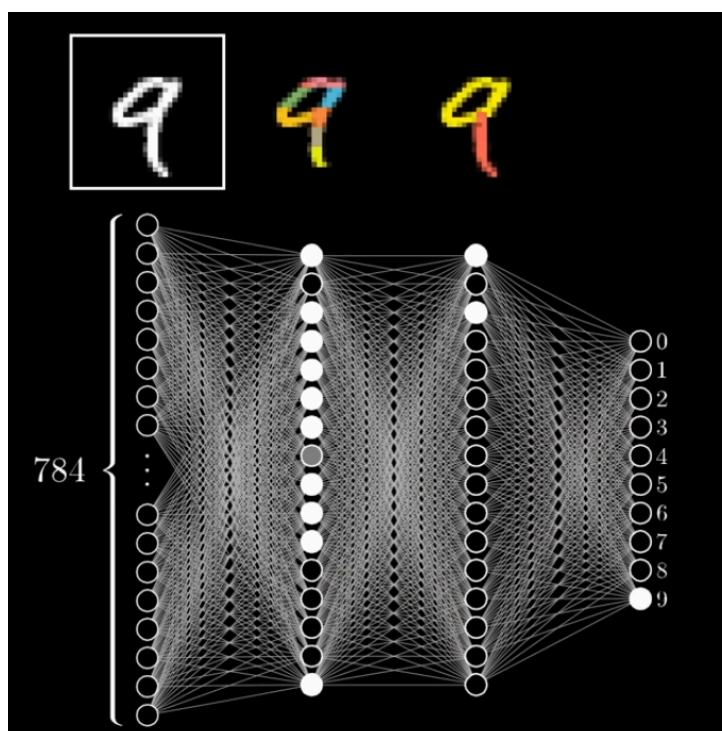
1° Camada → Inputs (Pixels da imagem). É uma imagem 27x27 (729 pixels, 729 inputs)

2° Camada → Reconhecimento de pequenos padrões. Reconhece pequenas retas e curvas, onde cada tipo de reta/curva é uma feature, representada por um único neurônio.

3° Camada → Junta todos os padrões anteriores encontrados (retas e curvas) para formar features mais complexas. Agora é capaz de reconhecer círculos e retas grandes.

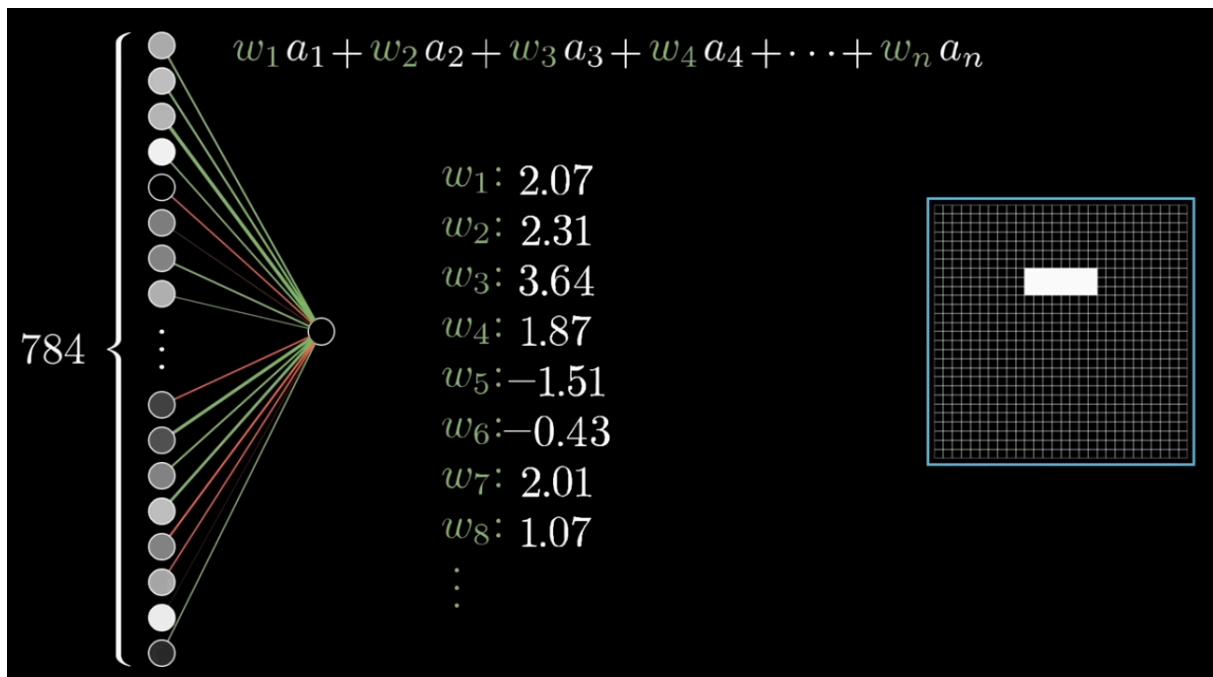
4° Camada → Camada de classificação. Utiliza os padrões encontrados pela 3° camada para classificar.

Cada camada reconhece padrões com base nas camadas anteriores, de modo que a camada seguinte reconhecerá padrões mais complexos, pois a camada atual já reconheceu alguns padrões pequenos.



1.2 Neurônio

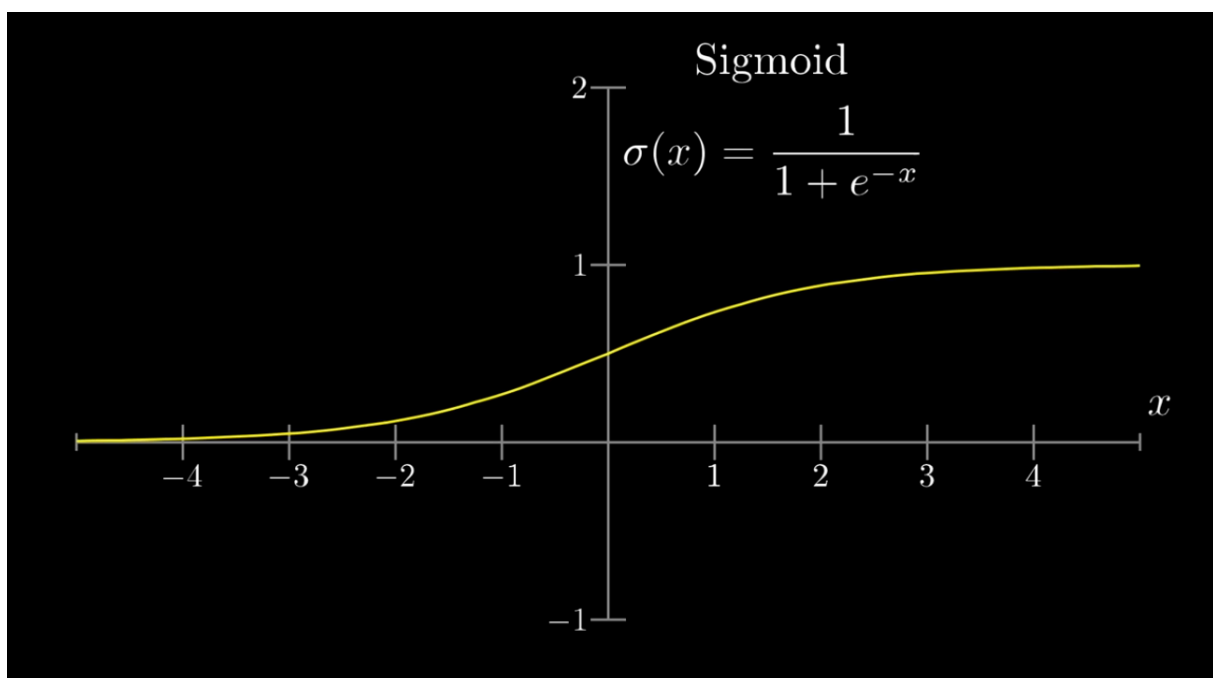
Cada neurônio coleta como input TODAS as saídas da camada anterior, e aplica pesos.



A ideia é que, mudando os pesos, os neurônios consigam enfatizar mais certas features específicas dos inputs.

A saída do neurônio não será um número grande. Usamos a função sigmoide (ou curva logística) para forçar o valor entre 0 e 1.

$$\sigma = \frac{1}{1 + e^{-x}}$$



Assim, essa função será aplicada à saída do neurônio.

$$S = \sigma(w_1 a_1 + \dots + w_n a_n)$$

E não para por aí. Bom, o neurônio ativará quando o $S = 0$, pois o resultado será 0.5. Mas, se você quer que o neurônio ative somente a partir de $S = 10$, adicionamos um bias:

$$S = \sigma(w_1 a_1 + \dots + w_n a_n - 10)$$

Assim:

- **Pesos:** Extraem característica com todos os inputs da camada anterior
- **Sigmoide:** Normaliza resultado (entre 0 e 1). Ativação do neurônio ocorre a partir de 0.5
- **Bias:** Controla quando o neurônio ativa.

2 Modelo matemático de output para a rede neural

2.1 Para um único neurônio

Para o primeiro neurônio da 2ª camada, por exemplo, teremos a seguinte função:

$$S = \sigma \left(w_1^{21} \cdot \underbrace{\sigma(w_1^{11} a_1 + \dots + w_n^{11} a_n - b_{11})}_{\text{Saída 1º neurônio, 1º camada}} + \dots + w_n^{2n} \cdot \underbrace{\sigma(w_1^{1n} a_1 + \dots + w_n^{1n} a_n - b_{1n})}_{\text{Saída último neurônio, 1º camada}} \right)$$

De modo matricial ($1 \times M, M \times N, N \times 1$):

$$S_{1 \times 1}^{21} = \sigma \left(\underbrace{\begin{pmatrix} w_1^{21} & \dots & w_n^{21} \end{pmatrix}}_{\text{Pesos neurônio}} \underbrace{\left(\begin{pmatrix} w_1^{11} & \dots & w_n^{11} \\ \vdots & \ddots & \vdots \\ w_1^{1m} & \dots & w_n^{1m} \end{pmatrix} \begin{pmatrix} a_1 \\ \dots \\ a_n \end{pmatrix} - \begin{pmatrix} b^{11} \\ \dots \\ b^{1m} \end{pmatrix} \right)}_{\substack{\text{1º camada} \\ \text{Input}}} - \underbrace{\begin{pmatrix} b^{21} \end{pmatrix}}_{\text{bias neurônio}} \right)$$

Note que a saída será um único número ($1 \times M, M \times N, N \times \Rightarrow \times 1$). Esse é o output do neurônio.

2.2 Para toda a camada

Se ampliarmos para todos os neurônios da 2ª camada:

$$S_{m \times 1}^2 = \sigma \left(\underbrace{\begin{pmatrix} w_1^{21} & \dots & w_n^{21} \\ \vdots & \ddots & \vdots \\ w_1^{2m} & \dots & w_n^{2m} \end{pmatrix}}_{\text{Pesos}} \underbrace{\left(\begin{pmatrix} w_1^{11} & \dots & w_n^{11} \\ \vdots & \ddots & \vdots \\ w_1^{1m} & \dots & w_n^{1m} \end{pmatrix} \begin{pmatrix} a_1 \\ \dots \\ a_n \end{pmatrix} - \begin{pmatrix} b^{11} \\ \dots \\ b^{1m} \end{pmatrix} \right)}_{\substack{\text{1º camada} \\ \text{Input}}} - \underbrace{\begin{pmatrix} b^{21} \\ \dots \\ b^{2m} \end{pmatrix}}_{\text{bias}} \right)$$

2º camada

Podemos simplificar para:

$$S_{m \times 1}^2 = \sigma(W_2 \times \sigma(W_1 A - B_1) - B_2)$$

Onde:

- $\sigma : R \rightarrow R = (1)(1 + e^{-x})$ (Função sigmoide)
- $W_k \rightarrow$ Matriz de pesos da camada K. Cada linha é um neurônio.
- $A \rightarrow$ Input
- $B_k \rightarrow$ Matriz de bias da camada K. Cada linha é um neurônio.

2.3 Para toda a rede

Note que cada camada consiste em uma única função:

$$S_k = \sigma(W_k S_{\{k-1\}} - B_k), S_0 = A$$

Onde:

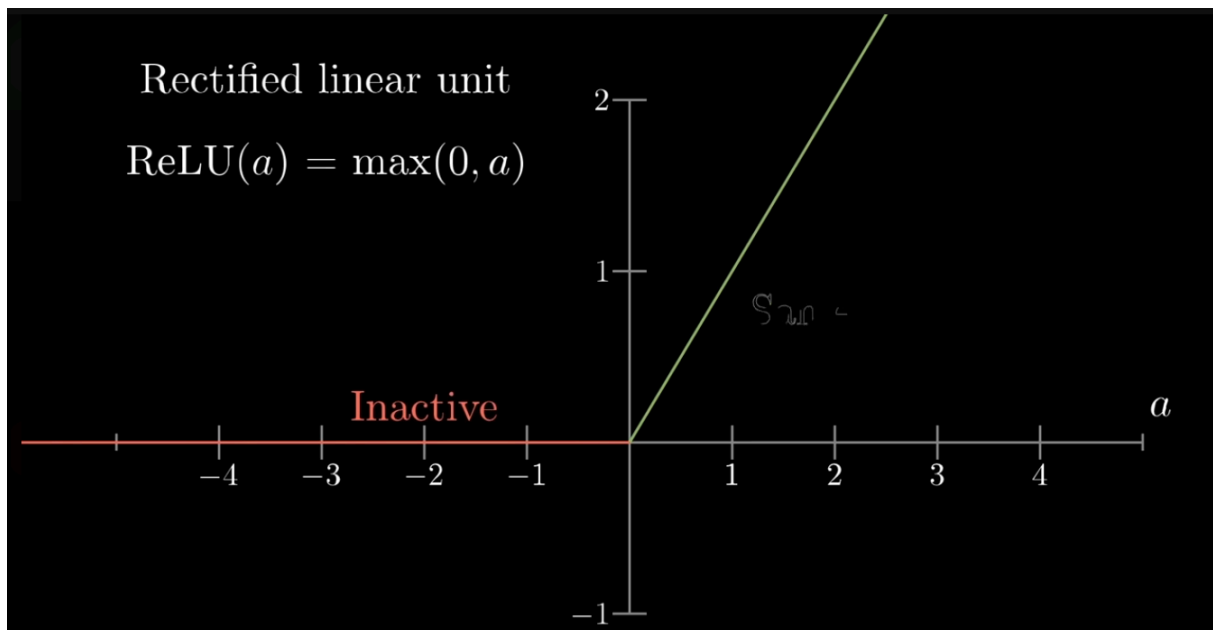
- $S_k \rightarrow$ Vetor de output dos M neurônios da camada anterior.
- $\sigma : R \rightarrow R = (1)(1 + e^{-x})$ (Função sigmoide) $\rightarrow$ Pode ser trocada por outra função qualquer
- $W_k \rightarrow$ Matriz de pesos da camada K. Cada linha é um neurônio. Possui N pesos/colunas
- $A \rightarrow$ Vetor de input (N entradas)
- $B_k \rightarrow$ Vetor de bias da camada K. Cada linha é um neurônio.

2.4 ReLU: Opção melhor do que Sigmoide

As sigmoides não são usadas mais, pois tornam mais difícil o treinamento. Atualmente, é usada uma função mais certa, a ReLU.

$$\text{ReLU}(x) = \max(0, x)$$

Sim, é simples assim. Desse modo, se $x > 0$, o neurônio será ativado.



As ReLU são utilizadas somente nas camadas intermediárias, pois propagam bem as features.

2.5 Camada final: Sem ReLU

Na camada final, precisamos de uma classificação binária (é/não é). Não faz sentido 0.91 quando se espera 1 ou 0. Assim, usamos uma função de ativação binária.

A ReLU é usada somente para hidden layers, para que os outputs possam transferir as features adequadamente.

Abaixo usaremos o Erro Quadrático Médio (MSE) para o treinamento. Portanto, a melhor função para a última camada com essa função custo é a Sigmoid.

Sigmoid

Função que “espreme” qualquer valor real para o intervalo (0,1)

$$S(z) = \frac{1}{1 + e^{-z}}$$

2.5.1 Problema matemático (saturação) com sigmoid

Ao usar MSE com sigmoid, a derivada na fórmula de ajuste torna-se um problema:

$$S' = S \cdot (1 - S)$$

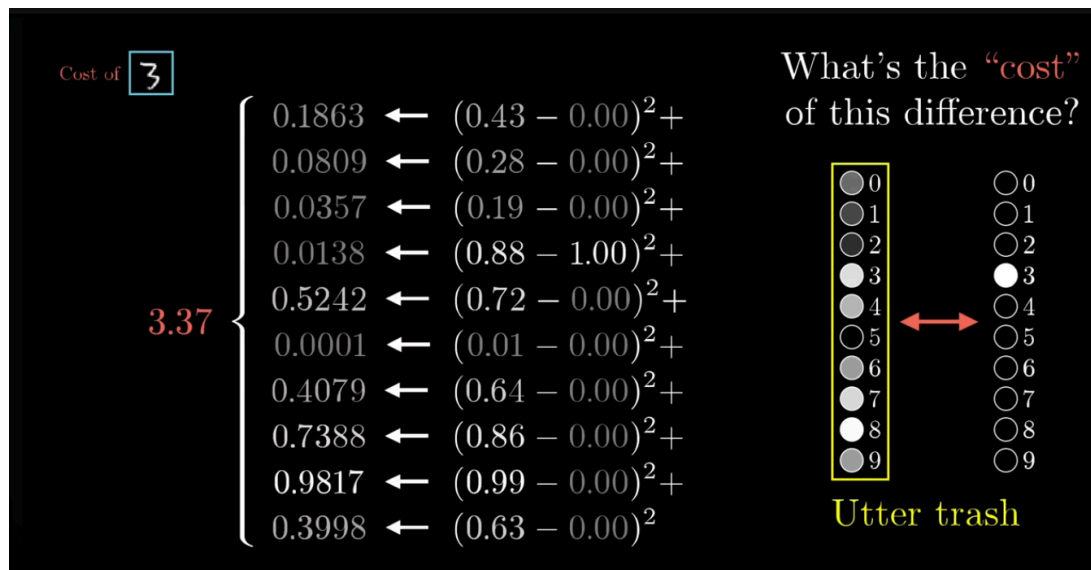
Isso significa que se o neurônio estiver “muito convicto” (saída perto de 0 ou 1), o valor de S' será quase zero. Isso significa que o ajuste dos pesos será minúsculo, mesmo que o erro seja grande. Isso é chamado de vanishing gradient.

Objetivo	Função de Saída	Função de Custo
Classificação	Softmax	Cross-Entropy
Regressão (prever)	Linear	MSE (Medium Square Error)
Classificação Binária	Sigmoid	Binary Cross-Entropy

3 Como a rede neural aprende

3.1 Custo do erro (módulo do vetor erro)

O custo do erro de uma rede neural é a soma do quadrado das diferenças entre o vetor resultado e o vetor rótulo.



Na verdade, isso é o módulo do vetor erro:

- B -> Vetor Rótulo
- A -> Vetor Resultado
- $\|\vec{AB}\|$ -> Módulo do vetor erro (diferença)

Esse custo pode ser tratado como uma função que recebe uma função de milhares de parâmetros (pesos sendo aplicados a outputs da camada anterior em cascata) e cospe um único número.

$$C(w_1, w_2, \dots, w_k) = e$$

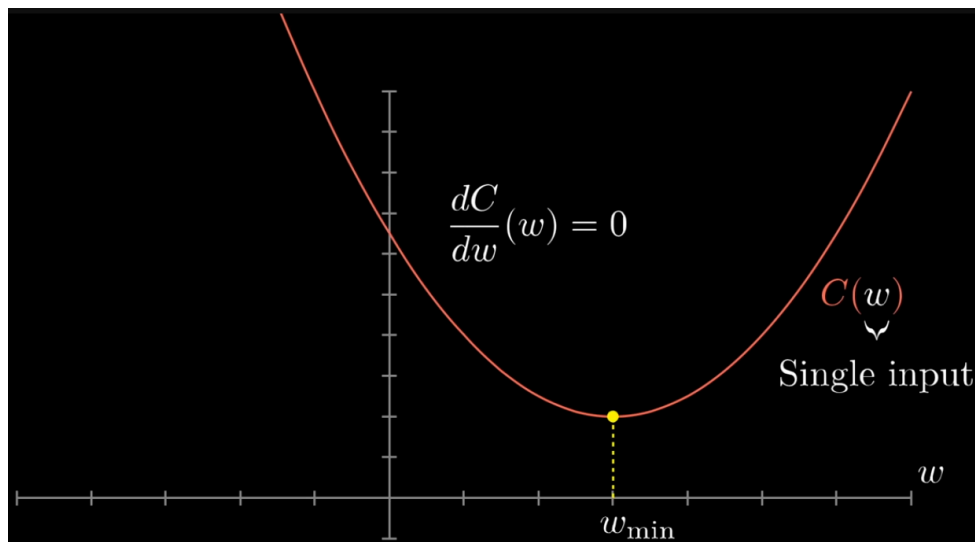
Na verdade, existem muitas funções custo diferentes. Essa é a **função custo MSE (Medium Square Error)**.

3.2 Achando peso ideal em $\mathbb{R}^1$

Mas vamos imaginar que temos um único peso na rede inteira. Assim:

$$C(w) = (w_1 a_1 - y)^2$$

E, se fazermos um gráfico $w \times y$, veremos que o erro pode ser minimizado em um ponto:



Isso nos ajuda a achar o valor ideal para esse peso!

Mas nossa função real de custo leva muito mais parâmetros do que isso, o que nos leva a $\mathbb{R}^n$.

3.3 Achando peso ideal em $\mathbb{R}^n$ com Função Custo MSE

3.3.1 Introdução

Agora, temos que levar em conta todos os pesos:

$$\vec{C}(w_1, w_2, \dots, w_k) = (S - Y)^2$$

Como podemos achar os pesos ideais que minimizam este erro? Bom, podemos tratar os pesos como dimensões, e aplicar Cálculo II (Cálculo com muitas variáveis) para encontrar o mínimo global.

3.3.2 Gradiente e gradiente descendente

Gradiente ∇

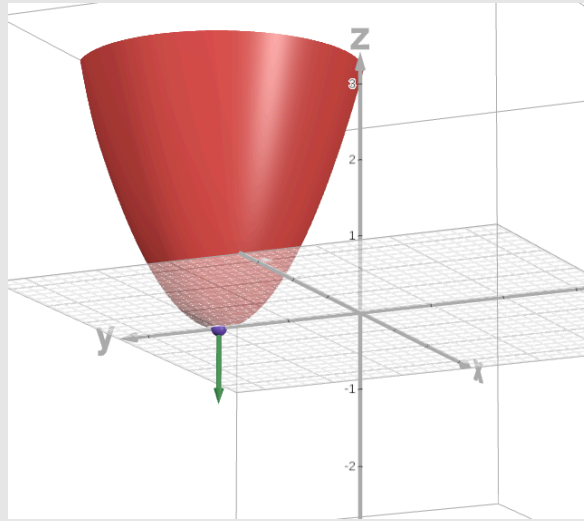
Lembra do gradiente? Ele leva de $\mathbb{R}^1 \rightarrow \mathbb{R}^n$.

$$\nabla[f(p_1, p_2, \dots, p_n) : \mathbb{R}^n \rightarrow \mathbb{R}^1] = \left(\partial \frac{f}{\partial p_1}, \partial \frac{f}{\partial p_2}, \dots, \partial \frac{f}{\partial p_n} \right)$$

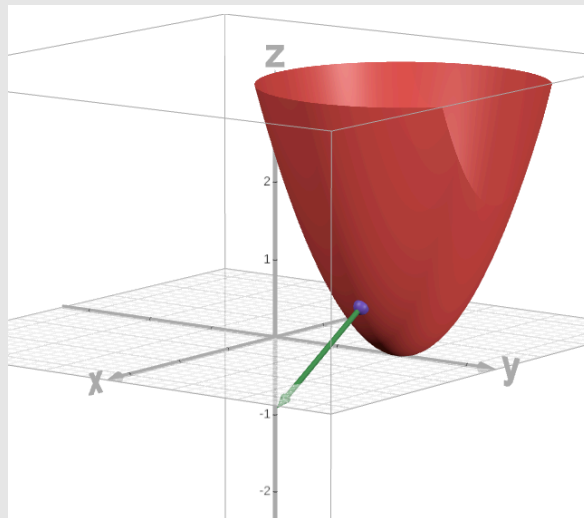
Pela definição, o vetor gradiente ∇f :

- Aponta para a direção de maior crescimento de f
 - Do mesmo modo, $-\nabla f$ aponta para a direção de maior decréscimo (gradiente descendente).
- Se $\nabla f = \vec{0}$, não importa onde você vá, o crescimento/decréscimo sempre será o mesmo. Nesse ponto, caímos em um mínimo/máximo local (mesmo conceito de derivada = 0 em $\mathbb{R}^2$)
 - Se for mínimo: Não importa a direção, f decresce igualmente
 - Se for máximo: Não importa a direção, f decresce igualmente
 - Em uma tigela, o mínimo global é o fundo.

- É perpendicular às curvas de nível ($f(p_1, \dots, p_n) = K$, superfícies de dimensão $n - 1$ onde a função possui sempre o mesmo valor).
- Sua magnitude representa o “tamanho” desse crescimento.

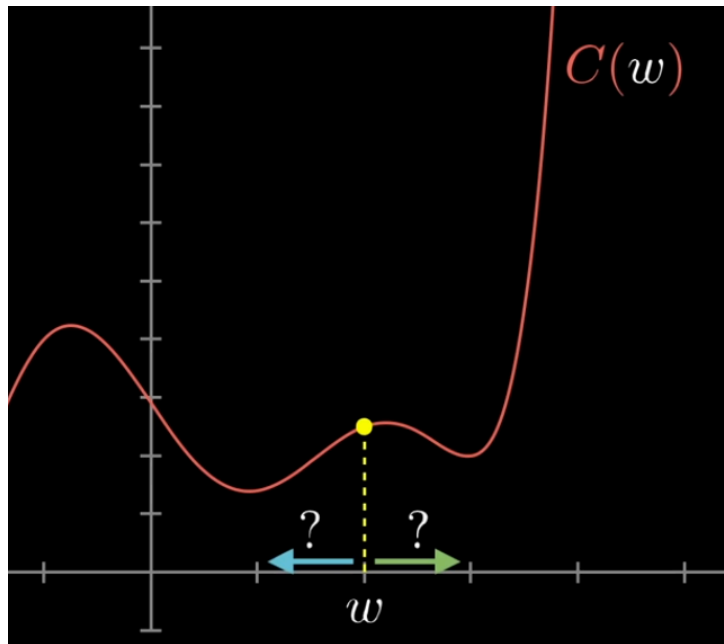


Aqui, encontramos um mínimo local, com $f(p_1, \dots, p_n) = z$



Aqui, o gradiente nos diz que se formos naquela direção, a função vai crescer mais do que em qualquer outra direção.

Para ajustar os pesos, o gradiente vai nos ajudar a saber qual direção minimiza o erro.

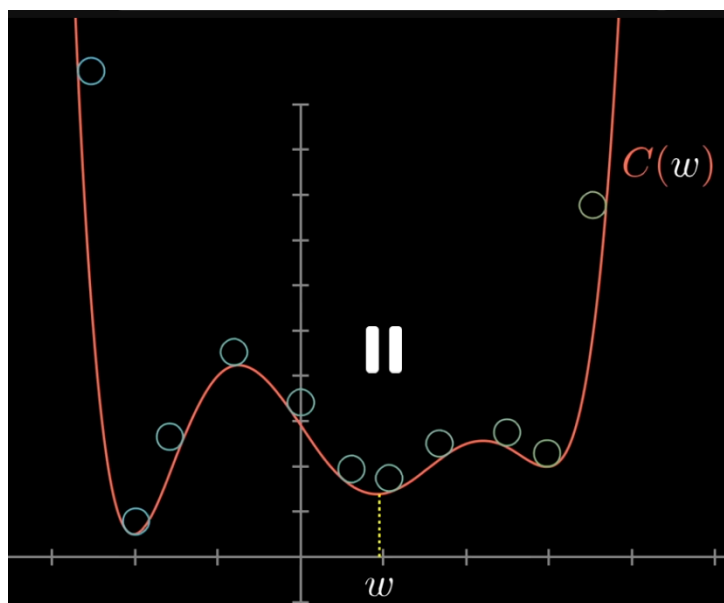


Aqui, o gradiente (derivada comum, pois estamos em R^2) apontará para a direita, que é onde a função cresce mais. Portanto, devemos ir na direção $-\nabla f$.

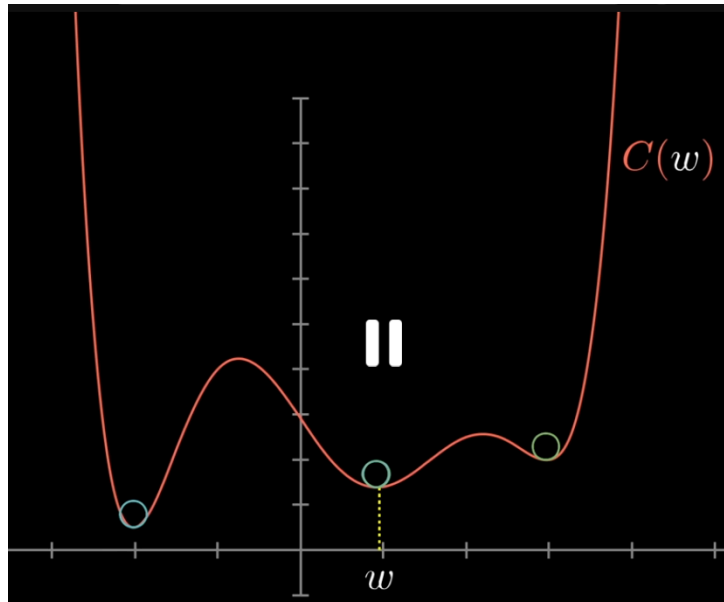
O conceito de gradiente descendente consiste em fazer vários ajustes nesse peso (não um ajuste único) para a direção de maior decrescimento, de modo a manter ele lá.

E não importa quão aleatório seja o peso inicial, ele sempre vai ser mantido em um mínimo local.

Antes:



Depois:



3.3.3 Aplicando gradiente descendente à Função Custo MSE

Vamos modelar a função custo de modo não matricial. Para um único neurônio da 2ª camada...

$$S_{21} = \sigma(w_1^{21} \cdot S_{11} + \dots + w_n^{21} \cdot S_{1n} - b_{21}) \Rightarrow W_2 \times S_1 - b_2$$

E a função custo para esse único neurônio:

$$\vec{C}(w_1^{21}, \dots, w_n^{21}) = (S - Y)^2 \quad (\text{Função Custo Total})$$

$$C = (S_{21} - y)^2$$

$$C = S_{21}^2 - 2yS_{21} + y^2$$

Obs: Vou definir:

$$\sigma(W_k) = S_k$$

Assim, temos uma função final, para todos os neurônios da 2ª camada:

$$C(S_{21}, y) = S_{21}^2 - 2yS_{21} + y^2$$

Vamos obter o primeiro termo do gradiente (a função de normalização tem que ser derivada também!)

Calculando as derivadas individuais:

$$\frac{d(S_{21}^2)}{dw_1^{21}} = 2 \frac{d(S_{21})}{dw_1^{21}} = 2\sigma'(W_{21}) \frac{d(W_{21})}{dw_1^{21}} = 2\sigma'(W_{21})S_{11}$$

$$\frac{d(2yS_{21})}{dw_1^{21}} = 2y\sigma'(W_{21}) \frac{d(W_{21})}{dw_1^{21}} = 2y\sigma'(W_{21})S_{11}$$

$$\frac{d(y^2)}{dw_1^{21}} = 0$$

Por fim:

$$\frac{dC}{dw_1^{21}} = 2S_{21} \cdot \sigma'(W_{21}) \cdot S_{11} - 2y \cdot \sigma'(W_{21}) \cdot S_{11}$$

Assim, se queremos minimizar o erro nessa direção, basta somar os pesos ao gradiente descendente (negativo do gradiente), para andar na direção que minimiza o erro.

$$w_1^{21} = w_1^{21} - K \frac{dC}{dw_1^{21}}(S_{21}, y, A)$$

$$w_1^{21} = w_1^{21} - K(2S_{21} \cdot \sigma'(W_{21}) \cdot S_{11} - 2y \cdot \sigma'(W_{21}) \cdot S_{11})$$

$$w_1^{21} = w_1^{21} - 2K(S_{21} - y) \cdot \sigma'(W_{21}) \cdot S_{11}$$

Tcharam! Esse é o ajuste de peso de um único neurônio, por gradiente descendente. O termo K é o tamanho do “passo” que queremos dar, e é arbitrário. Se for muito grande, vamos passar muitas vezes do ponto ideal. Se for muito pequeno, serão necessárias muitas iterações.

3.3.4 Conclusão para última camada

O gradiente da última camada é:

$$\frac{\partial C}{\partial W} = \underbrace{2(S_n - Y) \odot S'_n}_{\delta} \cdot S_{\text{ant}}^T$$

Dimensões: $\delta_{n \times 1}$, $W_{m \times n}$, $S_{n \times 1}$, $S'_{n \times 1}$, $Y_{n \times 1}$. A transposta ao final faz com que haja produto externo ao invés de produto interno, e a dimensionalidade para W é mantida.

$$S_n = \sigma(W_n \cdot S_{n-1} - b)$$

$$S'_n = \sigma(W_n \cdot S_{n-1} - b)$$

Podemos generalizar o ajuste de peso na última camada (W = matriz de pesos agora):

$$W_n = W_n - K[(2(S_n - Y) \odot S'_n) \cdot S_{\text{ant}}^T]$$

Na forma matricial:

A primeira multiplicação é elemento a elemento ($\odot$), e não um produto escalar. Isso pois a derivada parcial afeta localmente cada peso somente, todos são independentes.

Note que o ajuste de uma camada depende da camada anterior. Assim, o ajuste é feito do fim ao início. Isso é chamado de backpropagation.

3.3.5 Camadas anteriores

E para as camadas anteriores, ao invés de fazermos $\frac{\partial C}{\partial W_n}$, fazemos $\frac{\partial C}{\partial W_{n-1}}$. Cuidado, eu tava tentando obter $\frac{\partial C}{\partial W_{n-1}}$ a partir de $\frac{\partial C}{\partial W_n}$, o que é errado, acaba derivando duas vezes.

Bom, fazendo isso para o layer $n - 1$, temos (Obs: S_{ant} seria do atual (penúltimo layer) e S_{ant^2} seria do anterior):

$$\frac{\partial C}{\partial W_{n-1}} = W^T \cdot \underbrace{\delta \odot S'_{\text{ant}}}_{\delta_{\text{ant}}} \cdot S_{\text{ant}}^T$$

Generalizando para qualquer hidden layer:

$$\frac{\partial C}{\partial W_i} = \delta_i \cdot S_{\text{ant}}^T$$

$$\delta_i = W_{i+1}^T \cdot \delta_{i+1} \odot S'_i$$

Para a primeira camada, $S_{\text{ant}} = \text{Input}$. Ah, e W é transposto, pois é o próximo W . Assim, temos que transpor para as linhas/colunas baterem (estamos voltando!)

OBS: $\delta_i = \text{Antes do ajuste}$.

E, assim como fizemos na última camada, o ajuste é dado por:

$$W = W - K \frac{\partial C}{\partial W}$$

Produto de Hadamard ($\odot$)

Multiplicação elemento a elemento.

Inner Product / Outer Product em Vetores

Outer Product:

$$\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}_{2 \times 1} \cdot (b_1 \ b_2)_{1 \times 2} = \begin{pmatrix} a_1 b_1 & a_1 b_2 \\ a_2 b_1 & a_2 b_2 \end{pmatrix}$$

Inner product:

$$(a_1 \ a_2)_{1 \times 2} \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}_{2 \times 1} = (a_1 b_1 + a_2 b_2)_{1 \times 1}$$

(Do outer product eu não sabia).

3.3.6 Resumo: Ajuste de peso em $\mathbb{R}^n$ (Função Custo: MSE)

3.3.6.1 Ajuste de peso para última camada

$$\frac{\partial C}{\partial W} = 2 \underbrace{(S_n - Y) \cdots S'_n}_{\delta} \cdot S_{\text{ant}}^T$$

$$W_n = W_n - k \frac{\partial C}{\partial W}$$

3.3.6.2 Ajuste de peso para hidden layers

$$\frac{\partial C}{\partial W_i} = \delta_i \cdot S_{\text{ant}}^T$$

$$\delta_i = W_{i+1}^T \cdot \delta_{i+1} \cdots S'_i$$

$$W_i = W_i - k \frac{\partial C}{(\partial W)_i}$$

3.4 Achando Bias ideal em $\mathbb{R}^n$ para Função Custo Qualquer

O Bias também deve ser ajustado (aprendido) com o tempo. É ele quem define quando o neurônio ativa ou não.

$$\frac{\partial C}{\partial b} = \frac{\partial C}{\partial S} \cdot \frac{\partial S}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Obs: $S = \sigma(z)$.

$$\delta_{\text{camada}} = \frac{\partial C}{\partial S} \cdot \frac{\partial S}{\partial z}$$

3.5 Achando Peso ideal em $\mathbb{R}^n$ para Função Custo qualquer

Para achar o ajuste de pesos, basta repetir o mesmo processo, mas com a Função Custo Escolhida.

1. Para a última camada: Dada a função custo $C(S, Y)$ e $S = W \times S_{\text{ant}} + b_2$, ache $\frac{\partial C}{\partial W}$. O δ é o termo de $\frac{\partial C}{\partial W}$ que multiplica a saída anterior.
2. Para as camadas anteriores: Ache $\frac{\partial C}{\partial W_{n-1}}$, depois $\frac{\partial C}{\partial W_{n-2}}$, e veja se pode se induzir um padrão. O delta é o termo de $\frac{\partial C}{\partial W_i}$ que multiplica a saída anterior.

No código:

1. **Forward Pass:** Gere a saída S da rede neural, passando por todas as camadas.
2. **Backward Pass:** Calcule os deltas e gradientes de uma só vez
3. **Update:** Mude as matrizes de pesos com os gradientes
4. **Update:** Mude os bias com os deltas, utilizando a taxa de aprendizado (k)